Partie 6
Structures classiques (le retour)

INFO410 – Structures de données classiques

1. Rappel

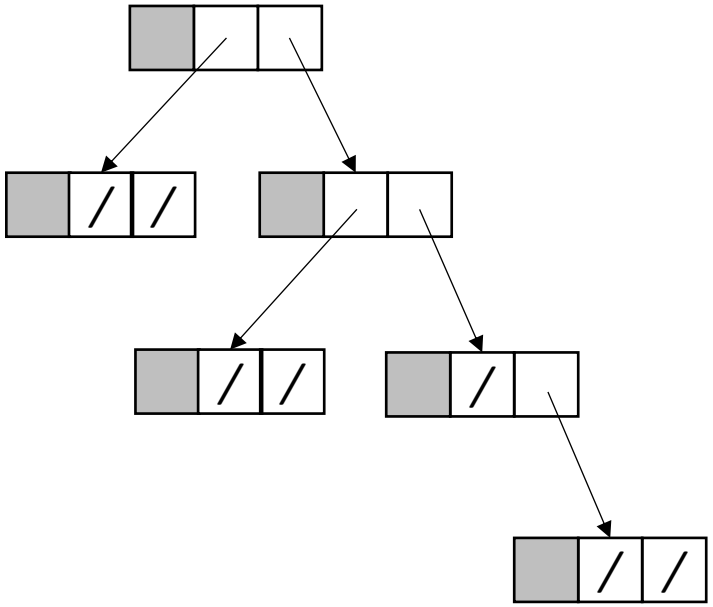
- On s'intéresse aux **collections homogènes** dont la taille peut **varier**
  - Ajouts ou retraits d’éléments
- On sait repérer une **extrémité** de la structure (ou les deux)
- On peut définir le **précédent** et/ou le **suivant** de tout élément qui n'est pas une extrémité
- On doit nécessairement parcourir **séquentiellement** la structure à partir d'une extrémité pour atteindre un élément particulier

2. Les arbres

- Ce sont des suites qui correspondent à la notion usuelle d'**arbres**
  - Exemple : avec des nœuds qui ont des fils qui sont des nœuds qui ont des fils…
- Dans le cadre de ce cours, nous utiliserons des **arbres binaires**, soient des arbres qui ont pour chaque nœud au maximum 2 fils, généralement 1 fils gauche et 1 fils droit
- Structures dynamiques **homogènes** à un seul point d'accès : la **racine**
- Un nœud qui ne possède pas de branche est appelé : une **feuille**

2. Les arbres

- Un arbre peut être représenté de cette manière



2. Les arbres

- Définition du type « `Arbre` »

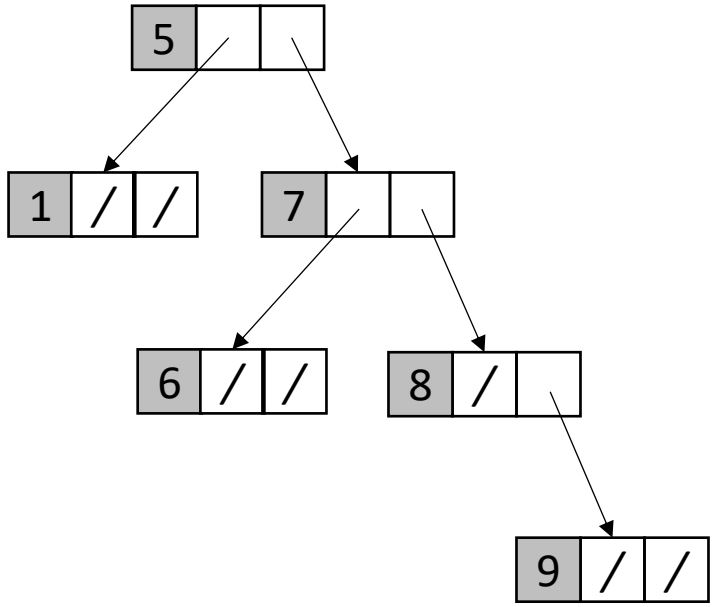
```
type noeud;
type arbre is access noeud;
type noeud is record
   info: T;
   fg, fd: arbre;
end record;
```

3. Exemple de parcours d’arbre

- Compte le nombre d’éléments dans un arbre

```
function compte(A: in arbre) return integer is
   …
end compte;
```

3. Exemple de parcours d'arbre

- Affichage des nœuds d'un arbre pour
  - 5 1 7 6 8 9
  - 1 5 6 7 8 9
  - 9 8 7 6 5 1

```
procedure affiche(A: in arbre) is
   …
end affiche;
```